

本地首頁偵錯 | Cloud-to-cloud | Google Home Developers

來源：<https://developers.home.google.com/codelabs/smarthome-debugging-local?hl=zh-tw>

步驟數：7

瞭解如何使用 GCP 指標和 Logging 找出並解決實際工作環境的問題。瞭解如何使用測試套件找出功能性與 API 問題。瞭解如何在開發本機首頁應用程式時使用 Chrome 開發人員工具。

目錄

1. 1. 事前準備
2. 2. 執行洗衣機應用程式
3. 3. 啟動智慧型洗衣機
4. 4. 測試本機住家控制應用程式
5. 5. 對 Local Home 應用程式偵錯
6. 6. 執行智慧型住宅測試套件
7. 7. 恭喜

步驟 1 · 約 0 分鐘

偵錯 Local Home

程式碼研究室簡介

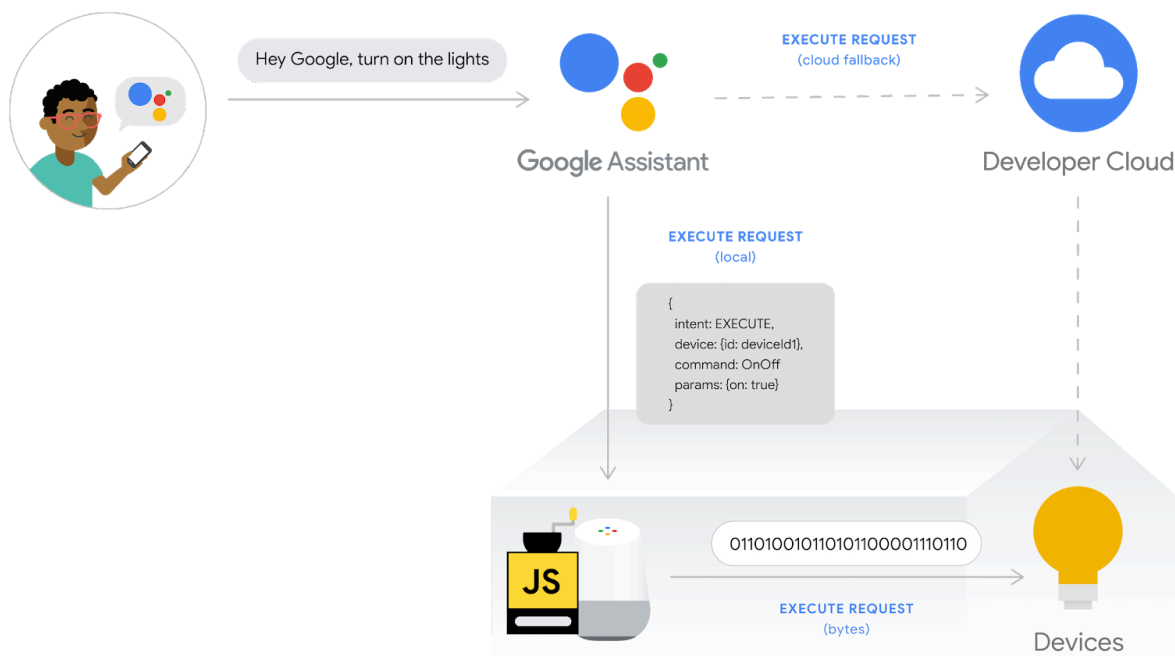
subject上次更新時間：5月 12, 2026

account_circle作者：Jessica Jong

1. 事前準備

智慧住宅整合功能可讓 Google 助理控制使用者住家中的連結裝置。如要建構雲端對雲端整合，您必須提供可處理[智慧住宅意圖](#)的雲端 Webhook 端點。舉例來說，當使用者說出「Ok Google，開燈」時，Google 助理會將指令傳送至雲端服務，更新裝置狀態。

Local Home SDK 會新增本機路徑，將智慧型住宅意圖直接轉送至 Google Home 裝置，藉此提升智慧型住宅整合程度，並在處理使用者指令時提高穩定性及縮短延遲時間。您可以使用 TypeScript 或 JavaScript 編寫及部署本機履行應用程式，在任何 Google Home 智慧音箱或 Google Nest 智慧螢幕上識別裝置並執行指令。然後，應用程式會使用現有的標準通訊協定，透過區域網路直接與使用者的現有智慧型裝置通訊，以執行指令。



如要建構品質優良的雲端對雲端整合，除錯是不可或缺的步驟。然而，如果沒有實用且易於使用的疑難排解和測試工具，除錯過程就會既困難又耗時。為方便偵錯雲端對雲端整合，Google Cloud Platform (GCP) [指標](#)、[記錄](#)和[智慧住宅測試套件](#)可協助您找出並解決整合問題。

必要條件

- [建立雲端對雲端整合開發人員指南](#)
- 執行「[為雲端整合啟用本機履行](#)」程式碼研究室

建構項目

在本程式碼研究室中，您將為雲端對雲端整合建構本機執行要求，並將其連結至 Google 助理，然後透過智慧型住宅測試套件和 Google Cloud Platform (GCP) 指標與記錄功能，偵錯 Google Home 應用程式。

課程內容

- 如何使用 GCP 指標和記錄檔找出並解決實際工作環境問題。

- 如何使用測試套件找出功能和 API 問題。
- 如何在開發區域網路居家控制應用程式時使用 Chrome 開發人員工具。

軟硬體需求

- 最新版 [Google Chrome](#)
- 已安裝 Google Home 應用程式的 iOS 或 Android 裝置
- Google Home 智慧音箱或 Google Nest 智慧螢幕
- [Node.js](#) 24 以上版本
- [Google 帳戶](#)
- Google Cloud [帳單帳戶](#)

注意：GitHub 存放區中有一些依附元件未列於此處。您必須遵循 GitHub 存放區 `package.json` 檔案中指定的依附元件版本。

2. 執行洗衣機應用程式

重要事項：開始前，請務必先執行[為雲端對雲端整合啟用店內取貨功能](#)程式碼研究室，並完成下列步驟：

- 在開發人員控制台中建立雲端對雲端整合
- 部署雲端履行服務，管理整合項目的虛擬智慧型洗衣機
- 部署與虛擬裝置通訊的本機住家應用程式

取得原始碼

點選下方連結，即可在開發機器上為本程式碼研究室下載範例：

...或者，您也可以從指令列複製 GitHub 存放區：



```
$ git clone https://github.com/google-home/smarthome-debug-local.git
```

注意：程式碼研究室存放區包含三個目錄：

-  `app-faulty`：起始程式碼，包含有錯誤的本機執行應用程式
-  `app-done`：完成的店內取貨應用程式完整程式碼
-  `virtual-device`：以 Node.js 編寫的模擬洗衣機裝置

關於專案

這個範例應用程式包含與「[為雲端整合啟用本機執行要求](#)」程式碼實驗室類似的子目錄和 Cloud 函式。但這裡不是 `app-start`，而是 `app-faulty`。首先，我們會從可運作但效能不佳的本機住家應用程式開始。

連結至 Firebase

我們會使用您在「[為雲端對雲端整合啟用本機履行功能](#)」程式碼研究室中建立的專案，但會部署在本程式碼研究室中下載的檔案。

前往 `app-faulty` 目錄，然後使用在「[為雲端對雲端整合啟用本機履行](#)」程式碼研究室中建立的整合專案，設定 Firebase CLI：

```
$ cd app-faulty
$ firebase use <project-id>
```

提示：如果您不知道 Cloud-to-Cloud 整合的專案 ID，可以在開發人員控制台的「專案首頁」下方找到「專案詳細資料」。

部署至 Firebase

前往 `app-faulty/functions` 資料夾，並使用 `npm` 安裝所有必要依附元件：

```
$ cd functions
$ npm install
```

注意：如果看到下列訊息，請忽略並繼續操作。這是因為部分舊版依附元件所致，詳情請參閱[這篇文章](#)。

```
found 5 high severity vulnerabilities
  run `npm audit fix` to fix them, or `npm audit` for details
```

前往 `app-faulty/local/` 目錄，然後執行下列指令，下載 TypeScript 編譯器並編譯應用程式：

```
$ cd ../local
$ npm install
$ npm run build
```

這會編譯 `index.ts` (TypeScript) 來源，並將下列內容放入 `app-faulty/public/local-home/` 目錄：

- `bundle.js`：編譯後的 JavaScript 輸出內容，包含本機應用程式和依附元件。
- `index.html`：用於放送應用程式的本機代管頁面，方便在裝置端進行測試。

安裝依附元件並設定專案後，即可首次執行應用程式。

```
$ firebase deploy
```

您應該會看到下列控制台輸出內容：

```
...
```

```
✓ Deploy complete!
```

```
Project Console: https://console.firebase.google.com/project/<project-id>/overview
Hosting URL: https://<projectcd -id>.web.app
```

這個指令會部署網頁應用程式，以及多個 [Cloud Functions for Firebase](#)。

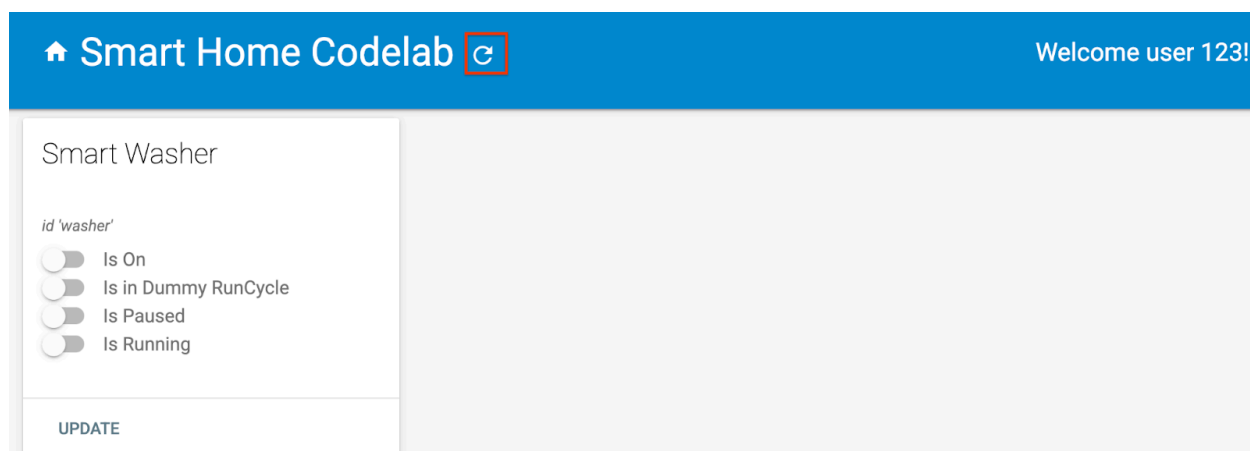
重要事項：建議重新啟動 Google Home 裝置，確保裝置載入您剛部署的本機住家應用程式。

更新 HomeGraph

在瀏覽器中開啟主機網址 (`https://<project-id>.web.app`)，即可查看網頁應用程式。在網頁 UI 中，按一下「重新整理」



按鈕，使用「要求同步」，以故障洗衣機應用程式的最新裝置中繼資料更新 HomeGraph。



開啟 Google Home 應用程式，確認洗衣機裝置是否顯示新名稱「故障洗衣機」。請記得將裝置指派到有 Nest 裝置的房間。



Codelab ▾

Invite home member 



Media



Settings

Office

2 devices



Nest Hub



Faulty Washer

Other Cast devices

5 devices



New Chromecast



Dining Room Hub



步驟 3 · 約 5 分鐘

3. 啟動智慧型洗衣機

如果您已完成「[為雲端整合啟用店面取貨功能](#)」程式碼研究室，應該已啟動虛擬智慧洗衣機。如果已停止，請記得重新啟動虛擬裝置。

啟動裝置

前往 `virtual-device/` 目錄並執行裝置指令碼，將設定參數做為引數傳遞：

```
$ cd ../../virtual-device
$ npm install
$ npm start -- \
  --deviceId=deviceid123 --projectId=<project-id> \
  --discoveryPortOut=3311 --discoveryPacket=HelloLocalHomeSDK
```

確認裝置指令碼是否使用預期參數執行：

```
(...): UDP Server listening on 3311
(...): Device listening on port 3388
(...): Report State successful
```

4. 測試本機住家控制應用程式

透過語音指令向 Google Home 裝置下達指令，例如：

「Ok Google，開啟洗衣機。」

「Ok Google，啟動洗衣機。」

「**Ok Google**，強制使用當地語言。」

「Ok Google，停止洗衣機。」

注意：Google 助理應在「force local」指令後回覆肯定語氣的訊息，例如「好的」或「瞭解」。「force local」指令會強制 Google 一律使用本機路徑，不會改用雲端路徑。

重要事項：說出「Ok Google，強制預設」即可停用「強制本機」指令。

在「強制本機」後嘗試控制洗衣機時，你會發現 Google 助理回應「抱歉，目前無法使用 Faulty Washer」。

這表示裝置無法透過本機路徑連線。發出「Ok Google，強制使用本機」指令前，裝置會使用雲端路徑，因為裝置無法透過本機路徑連線時，系統會改用雲端路徑。不過，在「強制本機」後，系統會停用回復雲端路徑的選項。

如要找出問題，請使用 Google Cloud Platform (GCP) [指標](#)和[記錄](#)，以及 Chrome 開發人員工具。

5. 對 Local Home 應用程式偵錯

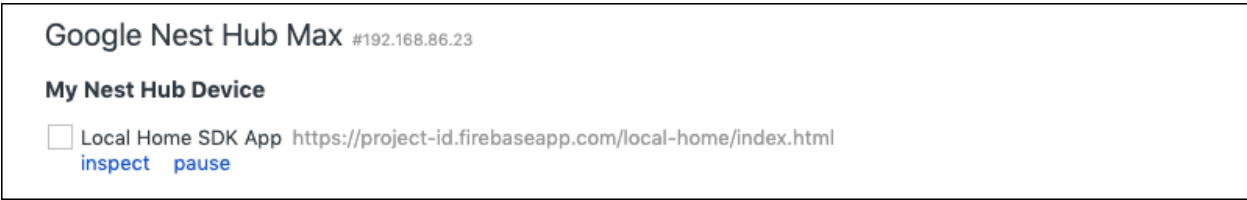
在下一節中，您將使用 Google 提供的工具，找出裝置無法透過本機路徑連線的原因。您可以使用 [Google Chrome 開發人員工具](#) 連線至 Google Home 裝置、查看控制台記錄，以及偵錯 Local Home 應用程式。您也可以將自訂記錄傳送至 [Cloud Logging](#)，瞭解使用者在 Local Home 應用程式中發現的主要錯誤。

連線 Chrome 開發人員工具

如要將偵錯工具連線至本機履行應用程式，請按照下列步驟操作：

1. 確認你已將 Google Home 裝置連結至具備 [Developer Console](#) 專案存取權的使用者。
2. 重新啟動 Google Home 裝置，這樣裝置就能取得 HTML 的網址，以及您在開發人員控制台中輸入的掃描設定。
3. 在開發機器上啟動 Chrome。
4. 開啟新的 Chrome 分頁，然後在網址欄位中輸入 `chrome://inspect`，啟動檢查器。

網頁上應會顯示裝置清單，而應用程式網址則會顯示在 Google Home 裝置名稱下方。



重要事項：如果沒有顯示網址，請取消連結並重新連結 Actions on Google 專案，然後重新啟動 Google Home 裝置。Play 管理中心的設定最多可能需要 20 分鐘才會生效。

啟動檢查器

按一下應用程式網址下方的「檢查」，啟動 Chrome 開發人員工具。選取「控制台」分頁，確認您可以看到 TypeScript 應用程式列印的 `IDENTIFY` 意圖內容。

提示：掃描時，Google Home 裝置會定期在網路上傳送探索封包。每次掃描結果都會觸發 `IDENTIFY` 意圖，直到應用程式驗證探索到的裝置 ID 為止。

```

IDENTIFY intent: {
  "requestId": "EA1BCB4B15EB5C94B57DFB19A940AAE7",
  "inputs": [
    {
      "intent": "action.devices.IDENTIFY",
      "payload": {
        "device": {
          "udpScanData": {
            "data": "6465766963656964313233"
          }
        },
        "structureData": {}
      }
    }
  ],
  "devices": [
    {
      "id": "washer",
      "customData": {}
    }
  ]
}
IDENTIFY handler: received local device id 6465766963656964313233
IDENTIFY response: {
  "intent": "action.devices.IDENTIFY",
  "requestId": "EA1BCB4B15EB5C94B57DFB19A940AAE7",
  "payload": {
    "device": {
      "id": "washer",
      "verificationId": "6465766963656964313233"
    }
  }
}
[ 0.267s] [smarthome.DeviceManager] Intent handler failed with error: DEVICE_VERIFICATION_FAILED
[ 0.274s] [smarthome.DeviceManager] VerificationId failed to match any unverified devices.
[ 0.285s] [smarthome.DeviceManager] Logging to stackdriver: {"requestId":"EA1BCB4B15EB5C94B57DFB19A940AAE7","timestamp":{"seconds":1615443081,"nanos":205000000},"appVersion":"1.0.0","intent":"action.devices.IDENTIFY","severity":"ERROR","message":{"requestId":"EA1BCB4B15EB5C94B57DFB19A940AAE7","payload":{"errorCode":"DEVICE_VERIFICATION_FAILED","debugString":"VerificationId failed to match any unverified devices."}}},"errorCode":"DEVICE_VERIFICATION_FAILED"}

```

這項輸出內容表示 IDENTIFY 處理常式已成功觸發，但 `IdentifyResponse` 中傳回的 `verificationId` 與 HomeGraph 中的任何裝置都不相符。讓我們新增一些自訂記錄，找出原因。

新增自訂記錄

雖然 Local Home SDK 會顯示 `DEVICE_VERIFICATION_FAILED` 錯誤，但對找出根本原因的幫助不大。讓我們新增一些自訂記錄，確保我們正確讀取及處理掃描資料。請注意，如果我們以錯誤拒絕 Promise，錯誤訊息實際上也會傳送至 [Cloud Logging](#)。

`local/index.ts`

```

identifyHandler(request: IntentFlow.IdentifyRequest):
  Promise<IntentFlow.IdentifyResponse> {
    console.log("IDENTIFY intent: " + JSON.stringify(request, null, 2));

    const scanData = request.inputs[0].payload.device.udpScanData;
    if (!scanData) {
      const err = new IntentFlow.HandlerError(request.requestId,
        'invalid_request', 'Invalid scan data');
      return Promise.reject(err);
    }

    // In this codelab, the scan data contains only local device id.
    // Is there something wrong here?
    const localDeviceId = Buffer.from(scanData.data);
    console.log(`IDENTIFY handler: received local device id
      ${localDeviceId}`);

    // Add custom logs
    if (!localDeviceId.toString().match(/^deviceid[0-9]{3}$/gi)) {
      const err = new IntentFlow.HandlerError(request.requestId,
        'invalid_device', 'Invalid device id from scan data ' +
        localDeviceId);
      return Promise.reject(err);
    }

    const response: IntentFlow.IdentifyResponse = {
      intent: Intents.IDENTIFY,
      requestId: request.requestId,
      payload: {
        device: {
          id: 'washer',
          verificationId: localDeviceId.toString(),
        }
      }
    };
    console.log("IDENTIFY response: " + JSON.stringify(response, null, 2));

    return Promise.resolve(response);
  }

```

此外，請變更本機 Google Home 應用程式版本，方便我們判斷是否使用正確版本。

local/index.ts

```
const localHomeSdk = new App('1.0.1');
```

新增自訂記錄後，您需要再次編譯應用程式，然後重新部署至 Firebase。

```
$ cd ../app-faulty/local
$ npm run build
$ firebase deploy --only hosting
```

現在請重新啟動 Google Home 裝置，載入更新後的本機居家應用程式。您可以在 Chrome 開發人員工具中查看**控制台**記錄，確認 Google Home 裝置是否使用預期版本。

```
Ready, App version: 1.0.1
Ready
```

```
smarthome_sdk.js:153
index.ts:151
```

存取 Cloud Logging

接下來，我們來看看如何使用 [Cloud Logging](#) 找出錯誤。如要存取專案的 Cloud Logging：

1. 前往 Cloud Platform 主控台的「Projects」（專案）頁面。
2. 選取智慧住宅專案。
3. 在「作業」下方，依序選取「記錄」>「記錄總管」。

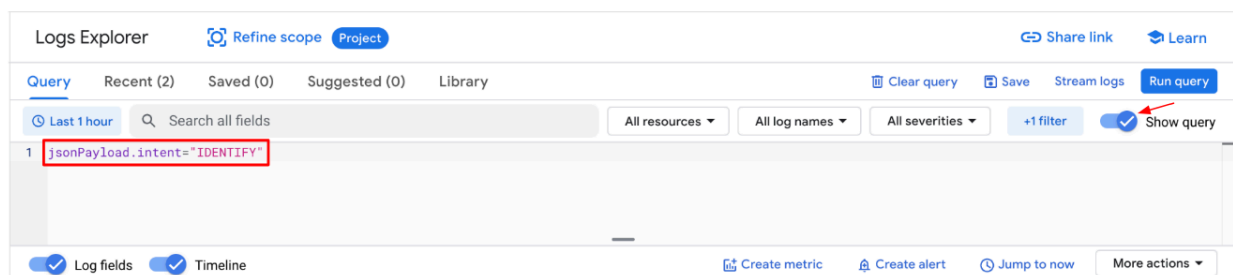
整合專案使用者的記錄資料存取權，是透過 Identity and Access Management (IAM) 管理。如要進一步瞭解記錄資料的角色和權限，請參閱 Cloud Logging [存取權控管](#)。

開發 Local Home 應用程式時，Chrome 開發人員工具是不可或缺的工具，但您必須是專案的開發人員，才能檢查裝置，而且只能檢查網路中的 Google Home 裝置。另一方面，Google Cloud Platform (GCP) [指標](#)和[記錄](#)可讓您查看所有使用整合服務的使用者指標和錯誤。建議您使用 GCP 指標和記錄，監控雲端對雲端整合的健康狀態。

使用進階篩選器

我們發現 `IDENTIFY` 意圖發生錯誤，因為系統無法識別本機裝置，導致本機路徑無法運作。不過，我們想確切瞭解問題，因此請先篩除 `IDENTIFY` 處理常式中發生的錯誤。

點選「顯示查詢」切換鈕，該按鈕會變成「查詢建立工具」方塊。在「Query builder」（查詢建立工具）方塊中輸入 `jsonPayload.intent="IDENTIFY"`，然後點選「Run query」（執行查詢）按鈕。



因此，您會取得 `IDENTIFY` 處理常式中擲回的所有錯誤記錄。接著，展開最後一個錯誤。您會在 `IDENTIFY` 處理常式中拒絕 Promise 時，找到剛才設定的 `errorCode` 和 `debugString`。

```
2021-03-11 14:39:52.696 UTC+8 {"requestId":"9BF6C343065F0C9E6B4CCEED47933607","payload":{"errorCode":"invalid_device","debugString":"Invalid device id from scan data: 6465766963656964313233"}}

{
  insertId: "170cm93g6t9jw3u"
  jsonPayload: {
    appVersion: "1.0.0"
    message:
      "{"requestId":"9BF6C343065F0C9E6B4CCEED47933607","payload":{"errorCode":"invalid_device","debugString":"Invalid device id from scan data: 6465766963656964313233"}}"
    timestamp: "2021-03-11T06:39:52.473Z"
    errorCode: "invalid_device"
    severity: "ERROR"
    requestId: "9BF6C343065F0C9E6B4CCEED47933607"
    intent: "IDENTIFY"
  }
  resource: {
    type: "assistant_action_project"
    labels: {1}
  }
  timestamp: "2021-03-11T06:39:52.696155029Z"
  severity: "ERROR"
  logName: "projects/debug-local-home-codelab/logs/assistant_smarthome%2Fintent_error_logs"
  receiveTimestamp: "2021-03-11T06:39:52.696155029Z"
}
```

從 `debugString` 中，我們可以得知本機裝置 ID 的格式不正確。Local Home 應用程式預期會取得以 `deviceid` 開頭的字串，後面接著 3 個數字，但這裡的本機裝置 ID 是十六進位字串。

修正錯誤

回到從掃描資料剖析本機裝置 ID 的原始碼，我們會發現將字串轉換為位元組時，我們未提供編碼。掃描資料會以十六進位字串的形式接收，因此呼叫 `Buffer.from()` 時，請將 `hex` 做為字元編碼傳遞。

local/index.ts

```

identifyHandler(request: IntentFlow.IdentifyRequest):
  Promise<IntentFlow.IdentifyResponse> {
    console.log("IDENTIFY intent: " + JSON.stringify(request, null, 2));

    const scanData = request.inputs[0].payload.device.udpScanData;
    if (!scanData) {
      const err = new IntentFlow.HandlerError(request.requestId,
        'invalid_request', 'Invalid scan data');
      return Promise.reject(err);
    }

    // In this codelab, the scan data contains only local device id.
    const localDeviceId = Buffer.from(scanData.data, 'hex');
    console.log(`IDENTIFY handler: received local device id
      ${localDeviceId}`);

    if (!localDeviceId.toString().match(/^deviceid[0-9]{3}$/gi)) {
      const err = new IntentFlow.HandlerError(request.requestId,
        'invalid_device', 'Invalid device id from scan data ' +
        localDeviceId);
      return Promise.reject(err);
    }

    const response: IntentFlow.IdentifyResponse = {
      intent: Intents.IDENTIFY,
      requestId: request.requestId,
      payload: {
        device: {
          id: 'washer',
          verificationId: localDeviceId.toString(),
        }
      }
    };
    console.log("IDENTIFY response: " + JSON.stringify(response, null, 2));

    return Promise.resolve(response);
  }

```

此外，請變更本機 Google Home 應用程式版本，方便我們判斷是否使用正確版本。

local/index.ts

```
const localHomeSdk = new App('1.0.2');
```

修正錯誤後，請編譯應用程式並重新部署至 Firebase。在 `app-faulty/local` 中執行：

```

$ npm run build
$ firebase deploy --only hosting

```


測試修正結果

部署完成後，請重新啟動 Google Home 裝置，載入更新後的本機住家應用程式。確認本機住家應用程式版本為 1.0.2，這次 Chrome 開發人員工具控制台應該不會顯示任何錯誤。

```
Ready, App version: 1.0.2
Ready
⚠ ▶ [ 0.083s] [smarthome.DeviceManager] Cast shell version: 1.54
⚠ ▶ [ 0.089s] [smarthome.DeviceManager] SDK version: 1.4.4
IDENTIFY intent: {
  "requestId": "749122D24E147042A76459060FE94723",
  "inputs": [
    {
      "intent": "action.devices.IDENTIFY",
      "payload": {
        "device": {
          "udpScanData": {
            "data": "6465766963656964313233"
          }
        }
      },
      "structureData": {}
    }
  ],
  "devices": [
    {
      "id": "washer",
      "customData": {}
    }
  ]
}
IDENTIFY handler: received local device id deviceid123
IDENTIFY response: {
  "intent": "action.devices.IDENTIFY",
  "requestId": "749122D24E147042A76459060FE94723",
  "payload": {
    "device": {
      "id": "washer",
      "verificationId": "deviceid123"
    }
  }
}
```

現在可以再次嘗試傳送指令給裝置。

「Ok Google，強制使用當地語言。」

「Ok Google，停止洗衣機。」

「Ok Google，開啟洗衣機。」

...

「Ok Google，強制預設。」

如果 Google 助理仍回覆「抱歉，洗衣機目前無法使用」，請再稍候幾分鐘，讓智慧住宅伺服器接收本機路徑。

重要事項：測試完成後，請記得說出「Ok Google，強制預設」。

6. 執行智慧型住宅測試套件

使用 Google Home 應用程式中的觸控手勢或語音指令驗證裝置後，即可使用自動化[智慧型住宅測試套件](#)，根據與整合項目相關聯的裝置類型和特徵驗證用途。測試套件會執行一系列測試，偵測整合功能中的問題，並針對失敗的測試案例顯示資訊豐富的訊息，讓您在深入研究事件記錄檔之前，就能加快偵錯速度。

執行智慧型住宅測試套件

請按照下列操作說明，透過測試套件測試雲端對雲端整合：

1. 在網路瀏覽器中開啟[智慧住宅測試套件](#)。
2. 按一下右上角的按鈕登入 Google。這樣一來，測試套件就能直接將指令傳送至 Google 助理。
3. 在「專案 ID」欄位中，輸入雲端對雲端整合的專案 ID。然後點選「下一步」繼續。
4. 在「測試設定」步驟中，「裝置和特徵」部分應會顯示「Faulty Washer」。
5. 由於範例洗衣機應用程式沒有新增 / 移除 / 重新命名洗衣機的 UI，請停用「測試要求同步」選項。在實際運作系統中，每當使用者新增 / 移除 / 重新命名裝置時，您都必須觸發「要求同步」。
6. 由於我們要測試本機和雲端路徑，請保持啟用「Local Home SDK」選項。
7. 按一下「下一步：測試環境」，開始執行測試。

Configure your test

Select test suites to run your test against.

Devices

Select the devices to test against.

☒	Device	Type	Software version	Hardware version	Device traits
☒	Faulty Washer	WASHER	1.0.1	1.0	OnOff, StartStop, RunCycle

Test settings

Report State and Request Sync

Time out*

20

seconds

Maximum 300 seconds

Configuration

- ☐ Test [Request Sync](#)
- ☐ Test [Scene](#)
- ☒ Supports Local Home SDK
 - ☒ Local Fulfillment ☐ Seamless Setup ☐ Supports Local Query
- ☐ Enable [secondary user verification](#)
If enabled, prompts for PIN/acknowledgement when needed
- ☐ Manual Verification

[Back: Select project](#)

[Next: Test environment](#)

測試完成後，您會發現本機路徑中的暫停/繼續測試失敗，而雲端路徑中的暫停/繼續測試則通過。

StartStop

- ✓ Start the Faulty Washer (local path)
- ✓ Query after 'Start the Faulty Washer'
- ✓ Check Report State and Query discrepancy
- ✗ Pause the Faulty Washer (local path)

```
AssertionError: Expected state to include: {"isPaused":true}, actual state:
{"on":true,"online":true,"currentRunCycle":
[{"currentCycle":"rinse","nextCycle":"spin","lang":"en"}],"currentTotalRemainingTime":1212,"currentCycleRemaini
ngTime":301,"isPaused":false,"isRunning":true,"activeZones":[]}: expected false to be true
```

- ✗ Query after 'Pause the Faulty Washer'

```
AssertionError: Expected state to include: {"isPaused":true}, actual state:
{"on":true,"online":true,"currentRunCycle":
[{"currentCycle":"rinse","nextCycle":"spin","lang":"en"}],"currentTotalRemainingTime":1212,"currentCycleRemaini
ngTime":301,"isPaused":false,"isRunning":true,"activeZones":[]}: expected false to be true
```

- ✓ Check Report State and Query discrepancy
- ✗ Resume the Faulty Washer (local path)

```
AssertionError: Expected state to include: {"isPaused":false}, actual state:
{"on":true,"online":true,"currentRunCycle":
[{"currentCycle":"rinse","nextCycle":"spin","lang":"en"}],"currentTotalRemainingTime":1212,"currentCycleRemaini
ngTime":301,"isPaused":true,"isRunning":false,"activeZones":[]}: expected false to be true
```

- ✓ Query after 'Resume the Faulty Washer'
- ✗ Check Report State and Query discrepancy

```
AssertionError: State 'isPaused' differs: queried 'false' on 2021-05-07T06:36:41.118105Z, reported 'true' on
2021-05-07T06:38:02.991897Z;
State 'isRunning' differs: queried 'true' on 2021-05-07T06:36:41.118105Z, reported 'false' on 2021-05-
07T06:38:02.991897Z: expected false to be true
```

- ✓ Stop the Faulty Washer (local path)
- ✓ Query after 'Stop the Faulty Washer'
- ✓ Check Report State and Query discrepancy

分析錯誤訊息

請仔細查看失敗測試案例中的錯誤訊息。這些狀態會顯示測試的預期狀態和實際狀態。在本例中，對於「暫停洗衣機」，預期狀態為 `isPaused: true`，但實際狀態為 `isPaused: false`。同樣地，對於「暫停洗衣機」，預期狀態為 `isPaused: true`，但實際狀態為 `isPaused: false`。

- ✗ Ok Google. Pause the Faulty Washer (local path)

```
AssertionError: Expected state to include: {"isPaused":true}, actual
state:
{"online":true,"on":true,"isPaused":false,"isRunning":true,"activeZones":
[],"currentRunCycle":
[{"currentCycle":"rinse","nextCycle":"spin","lang":"en"}],"currentTotalRem
ainingTime":1212,"currentCycleRemainingTime":301}: expected false to be
true
```

- ✗ Ok Google. Resume the Faulty Washer (local path)

```
AssertionError: Expected state to include: {"isPaused":false}, actual
state: {"isPaused":true,"isRunning":true,"activeZones":
[],"online":true,"currentRunCycle":
[{"currentCycle":"rinse","nextCycle":"spin","lang":"en"}],"currentTotalRem
ainingTime":1212,"currentCycleRemainingTime":301,"on":true}: expected
false to be true
```

從錯誤訊息來看，我們似乎在本地路徑中反向設定 `isPaused` 狀態。

找出並修正錯誤

讓我們找出 Local Home 應用程式將執行指令傳送至裝置的原始碼。`getDataCommand()` 是 `executeHandler()` 呼叫的函式，用於在傳送至裝置的執行指令中設定 `payload`。

local/index.ts

```
getDataForCommand(command: string, params: IWasherParams): unknown {
  switch (command) {
    case 'action.devices.commands.OnOff':
      return {
        on: params.on ? true : false
      };
    case 'action.devices.commands.StartStop':
      return {
        isRunning: params.start ? true : false
      };
    case 'action.devices.commands.PauseUnpause':
      return {
        // Is there something wrong here?
        isPaused: params.pause ? false : true
      };
    default:
      console.error('Unknown command', command);
      return {};
  }
}
```

我們確實是在反向狀態中設定 `isPause`，當 `params.pause` 為 `true` 時，應設為 `true`，否則應設為 `false`。因此，讓我們修正這個問題。

local/index.ts

```

getDataForCommand(command: string, params: IWasherParams): unknown {
  switch (command) {
    case 'action.devices.commands.OnOff':
      return {
        on: params.on ? true : false
      };
    case 'action.devices.commands.StartStop':
      return {
        isRunning: params.start ? true : false
      };
    case 'action.devices.commands.PauseUnpause':
      return {
        isPaused: params.pause ? true : false
      };
    default:
      console.error('Unknown command', command);
      return {};
  }
}

```

變更本機 Google Home 應用程式版本，以便我們判斷是否使用正確版本。

local/index.ts

```
const localHomeSdk = new App('1.0.3');
```

請記得再次編譯應用程式，然後重新部署至 Firebase。在 `app-faulty/local` 中執行：

```

$ npm run build
$ firebase deploy --only hosting

```

現在請重新啟動 Google Home 裝置，載入更新後的本機住宅應用程式。請確認本機住宅應用程式版本為 1.0.3。

測試修正結果

現在，請使用相同設定重新執行智慧住宅測試套件，您會發現所有測試案例都已通過。

再次執行測試套件前，請重新啟動虛擬智慧洗衣機，重設狀態。

Local Home SDK

Local Tests

Faulty Washer

OnOff

- ✓ Turn on the Faulty Washer (local path)
- ✓ Query after 'Turn on the Faulty Washer'
- ✓ Check Report State and Query discrepancy
- ✓ Turn off the Faulty Washer (local path)
- ✓ Query after 'Turn off the Faulty Washer'
- ✓ Check Report State and Query discrepancy

StartStop

- ✓ Start the Faulty Washer (local path)
- ✓ Query after 'Start the Faulty Washer'
- ✓ Check Report State and Query discrepancy
- ✓ Pause the Faulty Washer (local path)
- ✓ Query after 'Pause the Faulty Washer'
- ✓ Check Report State and Query discrepancy
- ✓ Resume the Faulty Washer (local path)
- ✓ Query after 'Resume the Faulty Washer'
- ✓ Check Report State and Query discrepancy
- ✓ Stop the Faulty Washer (local path)
- ✓ Query after 'Stop the Faulty Washer'
- ✓ Check Report State and Query discrepancy

RunCycle

- [Not Supported \(local path\)](#)

Cloud Tests

Faulty Washer

Device Requirements

- ✓ Required traits for device type WASHER

OnOff

- ✓ Turn on the Faulty Washer (cloud path)
- ✓ Query after 'Turn on the Faulty Washer'
- ✓ Check Report State and Query discrepancy
- ✓ Turn off the Faulty Washer (cloud path)
- ✓ Query after 'Turn off the Faulty Washer'
- ✓ Check Report State and Query discrepancy

StartStop

- ✓ Start the Faulty Washer (cloud path)
 - ✓ Query after 'Start the Faulty Washer'
 - ✓ Check Report State and Query discrepancy
-

步驟 7 · 約 0 分鐘

7. 恭喜



恭喜！您已成功學會如何使用智慧住宅測試套件和 Cloud Logging，排解本機 Google Home 應用程式的問題。

瞭解詳情

你也可以嘗試下列做法：

- 在裝置中新增更多[支援的特徵](#)，並使用測試套件進行測試。
- 在每個意圖處理常式中新增更多自訂記錄，並在 Cloud Logging 中查看。
- 建立[資訊主頁](#)、設定[快訊](#)，以及[以程式輔助方式存取指標資料](#)，取得整合的實用用量指標。

您也可以進一步瞭解如何[測試及提交](#)整合功能以供審查，包括將整合功能發布給使用者的認證程序。
